

BEGINNER-FRIENDLY GUIDE

YOLO26

Made Simple.

Learn Transfer Learning, Fine-Tuning and Incremental Learning from zero, with easy examples and real code.

What you will learn: how YOLO sees, how to teach it your own objects, and how to add new classes **without** it forgetting the old ones.

Save this post for later

Swipe →

What is Object Detection?

Three tasks. Same photo. Different questions.

CLASSIFICATION

"What is in this image?"

Output: one label. Example: *"helmet"*

DETECTION ← YOLO

"What is it, and where?"

Output: a box + a class + a confidence for each object.

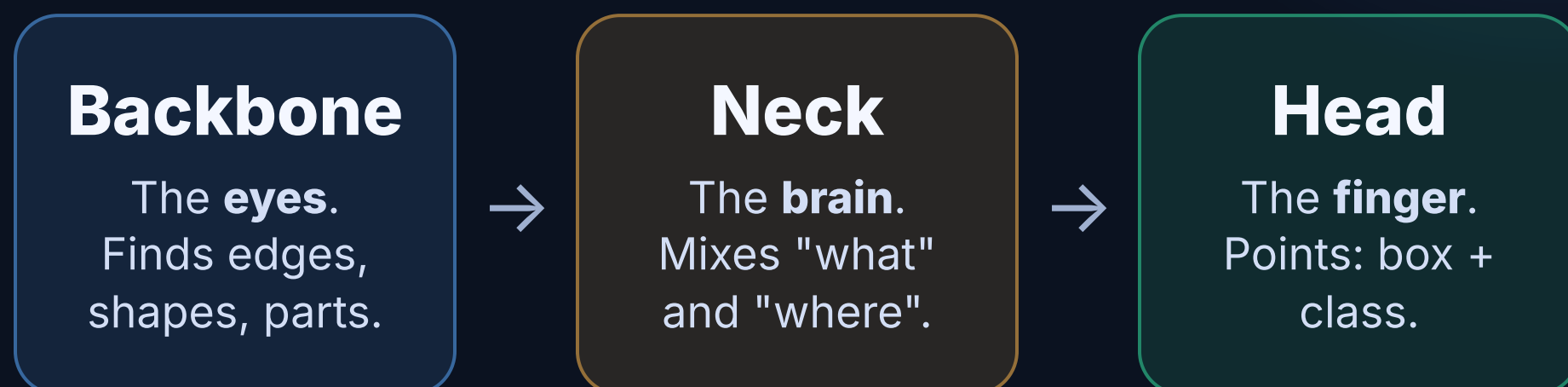
SEGMENTATION

"Which pixels belong to which object?"

Output: a mask for every object.

YOLO has 3 parts

Think of it like a person looking at a street.



Bonus: it sees at 3 scales

Small maps find small objects. **Big** maps find big objects. All in **one pass**, which is why YOLO is so fast.

Anchor-free, decoupled head

No hand-tuned "anchor boxes". Box and class are predicted in **separate branches**.

What changed in YOLO26

1

No NMS needed

One prediction per object. No extra "remove duplicate boxes" step.

2

DFL removed

A simpler head. Easier to export and run on small devices.

3

Better training

MuSGD optimizer, ProgLoss, and STAL (helps with small objects).

4

Built for the edge

Up to 43% faster on CPU than YOLO11n (ONNX), per Ultralytics docs.

Your code stays the same: `YOLO("yolo26s.pt")`

Swipe →

TRANSFER LEARNING

Don't start from **zero**.
Reuse what is **already**
learned.



If you can play **cricket**, learning **baseball** is easy. You already know how to hold a bat and watch the ball.

Backbone

Already trained on COCO.

REUSE

+

New head

Your classes.

TRAIN NEW

- ✓ Needs far less data
- ✓ Trains in minutes or hours
- ✓ Lower risk of overfitting

Transfer Learning in 5 lines

```
from ultralytics import YOLO

model = YOLO("yolo26s.pt") # pretrained
model.train(
    data="data.yaml",
    epochs=50, imgsz=640,
    patience=15,          # early stop
)
```

Little data? Freeze the backbone

Add **freeze=10** to train only the neck and head. Faster, and safer when you have fewer than ~500 images.

Use the best weights

Results are saved in **runs/.../weights/best.pt**. Always use **best.pt**, not last.pt.

FINE-TUNING

Adjust **slowly**, don't break what works

Fine-tuning is a way of doing transfer learning: unfreeze the model and train it with a **small learning rate**.

Stage 1

Warm-up

Backbone frozen.
New head learns.

LR = 0.01



Stage 2

Fine-tune

Unfreeze all.
Train gently.

LR = 0.001

COMMON TRAP

If `optimizer="auto"` (the default), Ultralytics ignores your `lr0`. Write `optimizer="SGD"` to control it.

Two-stage fine-tuning code

```
# Stage 1: backbone frozen
m = YOLO("yolo26s.pt")
m.train(data=D, epochs=20, freeze=10,
        optimizer="SGD", lr0=0.01,
        name="ft_stage1")

# Stage 2: unfreeze, small LR
m = YOLO("runs/.../ft_stage1/weights/best.pt")
m.train(data=D, epochs=40, freeze=0,
        optimizer="SGD", lr0=0.001,
        name="ft_stage2")
```

- ✓ Train loss down, **val mAP up** = good
- ✓ Train loss down, val loss up = **overfitting**

INCREMENTAL LEARNING

Teach a new class **without forgetting** the old ones

MODEL V1

helmet
vest

TRAIN ONLY ON GLOVES

gloves ✓
~~helmet vest~~

This is **catastrophic forgetting**. New learning overwrites old knowledge.

The hidden cause: missing labels

New images also contain helmets and vests, but they are **unlabeled**. The model learns they are "**background**" and stops detecting them.

3 steps that **work**

Pseudo-label

1

Let the **old model** label helmet and vest on the new images.

Replay

2

Mix in ~20–50% of the **old data** with the new data.

Small LR

3

Start from old weights, **lr0** $\approx$ **0.002**, then test all classes.

GOLDEN RULE

Never change old class IDs. Add the new class **at the end**:
0 helmet, 1 vest, **2 gloves**.

Which method **should I use?**

UNDER 500 IMAGES

Transfer Learning with **freeze=10** and a small model.

500 TO 5,000 IMAGES

Two-stage fine-tuning.

NEW CLASS LATER, OLD DATA AVAILABLE

Merge all data and fine-tune again. It is the cleanest way.

NEW CLASS LATER, NO OLD DATA

Incremental: pseudo-labels + replay + small LR.

THAT'S THE SIMPLE VERSION

Want the **full guide** with all the code?

- ✓ Setup and dataset labeling
- ✓ Full training and evaluation code
- ✓ Incremental learning scripts
- ✓ Export, troubleshooting, 8-week roadmap



Link in the first comment



Repost •



Save •



Follow for more